TP Kafka avec - Gestion de médicaments

L'objectif est de créer un petit projet de streaming de données avec Apache Kafka, Docker Compose et Python. Nous allons construire nous-mêmes le producteur (producer) et le consommateur (consumer).

Le contexte métier sera la gestion de médicaments .

Objectifs

- Lancer Kafka à l'aide de docker compose up.
- Créer un topic Kafka.
- Écrire un producer Python qui envoie des données au broker Kafka.
- Écrire un consumer Python qui lit les données .
- Tester le fonctionnement et observer les messages échangés.

Architecture

Le projet utilisera Docker Compose pour démarrer Kafka, puis trois programmes Python indépendants :

- Producer : lit les données des médicaments et les envoie à Kafka.
- Consumer 1 — consumer_pharmacy.py : reçoit les médicaments et affiche les informations de la commande.
- Consumer 2 — consumer_analytics.py : reçoit les mêmes événements et calcule des statistiques par pays et par prix.

Les deux consumers utiliseront des groupes Kafka différents afin que chacun reçoive tous les messages, indépendamment de l'autre.

tp-kafka-medicines/

Docker-compose.yml Démarrage du broker Kafka en mode KRaft.

Requirements.txt Dépendances Python.

data/

medicines.json Jeu de données initial en anglais.

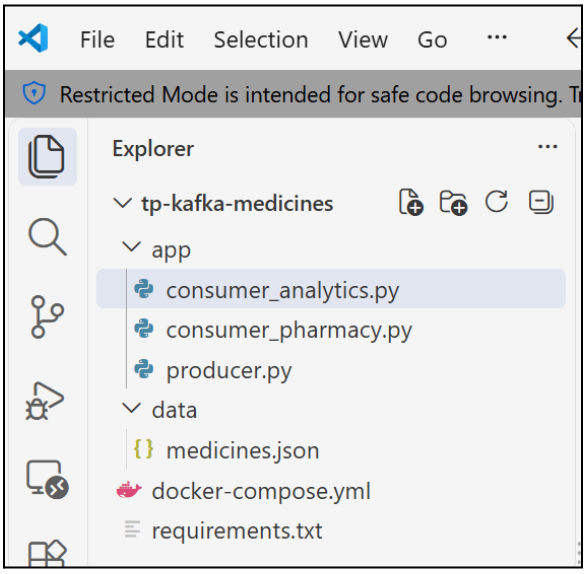
app/

producer.py Publie les événements dans Kafka.

Consumer_pharmacy.py Consumer 1 : traitement des commandes.

Consumer_analytics.py Consumer 2 : statistiques des ventes.

Notion importante : deux consumers appartenant au même groupe se répartissent les partitions. Ici, nous utilisons deux groupes distincts : chaque application peut donc lire l'ensemble des messages du topic.



2. Contexte métier

Une entreprise pharmaceutique souhaite diffuser les événements de commande de médicaments à plusieurs applications en temps réel.

Chaque médicament est représenté par les champs suivants :

Champ	Signification	Exemple
id	Identifiant du produit	101
name	Nom du médicament	Paracetamol

category	Catégorie du produit	Pain Relief
price	Prix unitaire	4.50
delivery_country	Pays de livraison	France

Exemple de message envoyé à Kafka :

```
{
  "id": 101,
  "name": "Paracetamol",
  "category": "Pain Relief",
  "price": 4.5,
  "delivery_country": "France"
}
```

Ce TP est un exercice technique avec des données fictives. Il ne couvre ni les prescriptions, ni les posologies, ni la gestion clinique des patients.

Partie 1 — Démarrer Kafka avec Docker Compose
Utilisez le fichier partagé sur Moodle.

Comprendre la configuration

- image : image Docker de Kafka.
- container_name : nom du conteneur.
- 9092:9092 : rend Kafka accessible depuis les programmes Python exécutés sur ta machine.
- KAFKA_PROCESS_ROLES : permet au même nœud de jouer les rôles de broker et de contrôleur KRaft.
- KAFKA_ADVERTISED_LISTENERS : indique aux clients l'adresse à utiliser.
- KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR : fixe à 1 le facteur de réplication du topic interne des offsets, ce qui convient à un broker unique.

Cette configuration convient à un TP local, mais elle ne fournit pas la sécurité ni la haute disponibilité nécessaires en production.

Étape 1 — Démarrer Kafka

Exécute à la racine : docker compose up -d

Vérifie le démarrage : docker compose ps

```
C:\Users\manel_xollxku\Documents\TPS>cd tp-kafka-medicines

C:\Users\manel_xollxku\Documents\TPS\tp-kafka-medicines> docker compose up -d
[+] up 11/13
 - Image apache/kafka:4.1.1 [################] 229.5MB / 232.1MB Pulling
```

Étape 2 — Créer le topic medicines

Un topic est une catégorie de messages Kafka. Tous les médicaments seront publiés dans le topic medicines.

docker exec kafka-medicines /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --create **--topic medicines** --partitions 1 --replication-factor 1

Les paramètres importants :

- docker exec exécute une commande dans le conteneur Kafka.
- --create crée le topic.
- --topic medicines définit son nom.
- --partitions 1 crée une partition.
- --replication-factor 1 conserve une seule copie sur notre broker.

Pour vérifier :

docker exec kafka-medicines /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic medicines

Si le topic existe déjà, conserve-le et passe à l'étape suivante.

Étape 3 — Préparer les dépendances Python

Dans requirements.txt :

confluent-kafka>=2.6,<3.0

- Crée un environnement virtuel : python -m venv .venv
- Active-le sous Windows PowerShell : .venv\Scripts\Activate.ps1
- Installe ensuite la bibliothèque : python -m pip install -r requirements.txt

```
C:\Users\manel_xollxku\Documents\TPS\tp-kafka-medicines>python -m pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting confluent-kafka<3.0,>=2.6 (from -r requirements.txt (line 1))
  Downloading confluent_kafka-2.16.0-cp313-cp313-win_amd64.whl.metadata (39 kB)
Downloading confluent_kafka-2.16.0-cp313-cp313-win_amd64.whl (4.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.7/4.7 MB 29.9 MB/s eta 0:00:00
Installing collected packages: confluent-kafka
Successfully installed confluent-kafka-2.16.0

[notice] A new release of pip is available: 24.3.1 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Nous pouvons maintenant passer à la création du producer et des deux consumers.

Partie 2 — Développer le Producer Python

Étape 2.1 — Créer le jeu de données

Dans data/medicines.json, ajoute les données suivantes :

```
[
 {
  "id": 101,
  "name": "Paracetamol",
  "category": "Pain Relief",
  "price": 4.50,
  "delivery_country": "France"
 },
 {
  "id": 102,
  "name": "Vitamin C",
  "category": "Vitamin",
  "price": 8.90,
  "delivery_country": "Belgium"
 },
 {
  "id": 103,
  "name": "Ibuprofen",
  "category": "Pain Relief",
  "price": 6.20,
  "delivery_country": "France"
 },
 {
  "id": 104,
  "name": "Antihistamine",
  "category": "Allergy",
  "price": 12.50,
  "delivery_country": "Spain"
 },
 {
  "id": 105,
```

```
    "name": "Cough Syrup",
    "category": "Cough and Cold",
    "price": 7.40,
    "delivery_country": "Belgium"
  }
]
```

Les prix sont des exemples fictifs exprimés en euros.

Étape 2.2 — Créer app/producer.py

Le producer va lire le fichier JSON, puis envoyer chaque médicament comme message indépendant dans le topic Kafka medicines.

Instructions que vous pouvez utiliser dans le code

Instruction	Fonction
Producer({...})	Initialise le client producteur.
json.load(file)	Lit le tableau JSON comme une liste Python.
json.dumps(medicine)	Transforme un dictionnaire en texte JSON.
key=str(medicine["id"])	Utilise l'identifiant comme clé Kafka.
producer.produce(...)	Place le message dans la file d'envoi.
producer.poll(0)	Traite les événements du client sans attente.
producer.flush(timeout=10)	Attend les messages en attente, au maximum 10 secondes.

La fonction **delivery_callback** indique si Kafka a confirmé la livraison. Un offset identifie la position d'un message dans sa partition.

Étape 2.4 — Exécuter le producer

Vérifie que Kafka est actif, puis lance : python app/producer.py

Tu devrais voir les cinq médicaments préparés et les confirmations de livraison.

Tu peux relancer le producer plusieurs fois, mais cela publiera à nouveau les cinq événements. Kafka ne supprime pas automatiquement les anciens messages lorsqu'un producer est relancé.

Étape 3.1 — Consumer 1 : app/consumer_pharmacy.py

Le premier consumer représente une application de gestion des commandes de la pharmacie. Pour chaque médicament reçu, il affiche les informations du produit.

Points à retenir sur le code à partager.

- group.id vaut pharmacy-group.
- subscribe([TOPIC]) abonne le consumer au topic medicines.
- poll(1.0) récupère les messages.
- json.loads() convertit le JSON reçu en dictionnaire Python.
- consumer.close() ferme proprement le consommateur.

Ce consumer ne calcule pas de statistiques : il traite chaque événement individuellement.

Étape 3.2 — Consumer 2 : app/consumer_analytics.py

Le deuxième consumer représente une application d'analyse des ventes. Il reçoit les mêmes événements, mais effectue un traitement différent :

- compter les médicaments reçus ;
- calculer la somme des prix ;
- compter les événements par pays de livraison ;
- afficher un résumé après chaque message.

RQ: Utilisez deux groupes différents

Consumer	**Groupe Kafka**	**Travail réalisé**
consumer_pharmacy.py	pharmacy-group	Affiche chaque médicament reçu.

consumer_analytics.py	analytics-group	Calcule les statistiques des événements.

Kafka gère séparément les offsets des deux groupes. Ainsi, le fait que le premier consumer ait lu un message ne signifie pas que le deuxième l'a déjà lu.

Étape 3.3 — Lancer les deux consumers en parallèle

Le test doit être réalisé avec trois terminaux ouverts dans le même dossier de projet.

Analysez et expliquez les résultats obtenus.